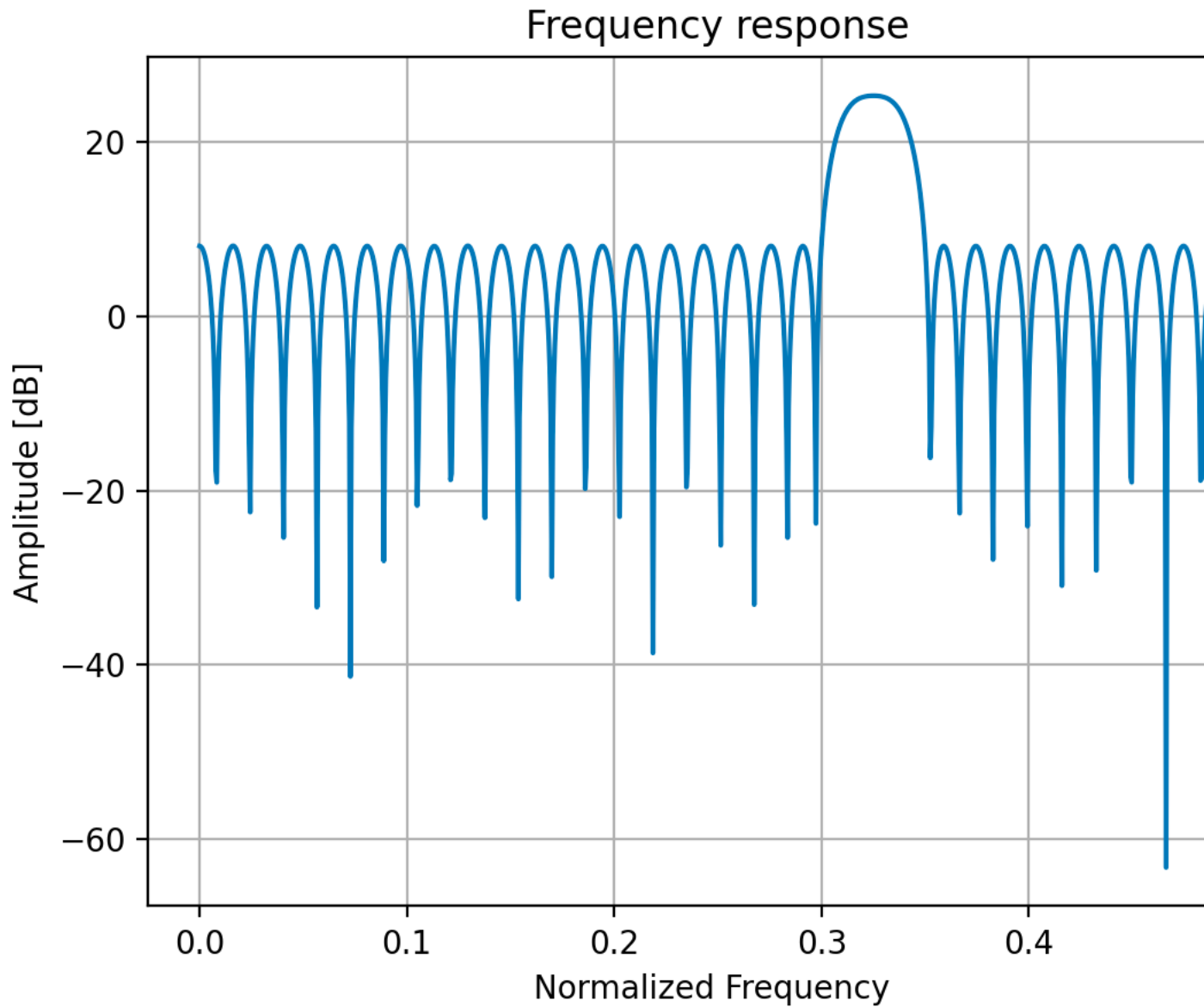


a)Après lecture de ce fichier en python, il s'agit d'un filtrage FIR (non recursive)

-> b)réponse fréquentielle du filtre



- caractéristique du filtre: on atténue d'environ **15 DB** entre

F1=0.00 et F2=0.29 puis entre F3= 0.35 et F4=0.5

Les fréquences sont normalisées (**fs=1**)

nombre de coef*64*

freq = 0.09765625 | H = (-1.888808451958261+1.6496184527290072j)

amplitude 2.52816088608128

Gain 8.056094158931844dB

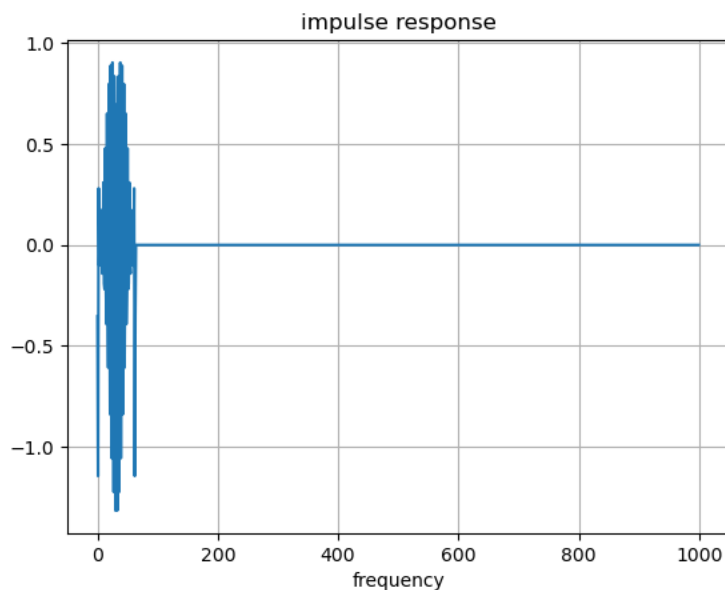
In []:

```
import numpy as np
import scipy.signal as sci
import matplotlib.pyplot as plt
import serial
import struct
import time
%matplotlib qt

coef=np.loadtxt("coef21.txt")
b=coef
a=1
#nombre de coeficientes
N = len(coef)
print('nombre de coef',N)
#print(coef)
M=1024 #number of samples for the simulation
imp=np.zeros(M)
imp[0]=1 #create an impulse signal
h=sci.lfilter(b,a,imp) #h is the impulse response of this filter
# réponse fréquentielle
H=np.fft.fft(h) # FFT
w,H= sci.freqz(b,a, worN=1024, fs=1) # frequency response
plt.figure()
plt.plot(w,10*np.log10(np.abs(H)**2)) # Power scale
print(f'freq = {w[200]} | H = {H[300]}')
T=np.abs(H[200])
print(f'amplitude {T}')
# Gain in dB
Gian=20*np.log10(T)
print(f'Gain {Gian} dB')
plt.title('Frequency response')
plt.xlabel('Normalized Frequency ')
plt.ylabel('Amplitude [dB]')
plt.grid()
```

c) teste du filtre sur les signaux de références différentes:

-> réponse impulsionnelle



In []:

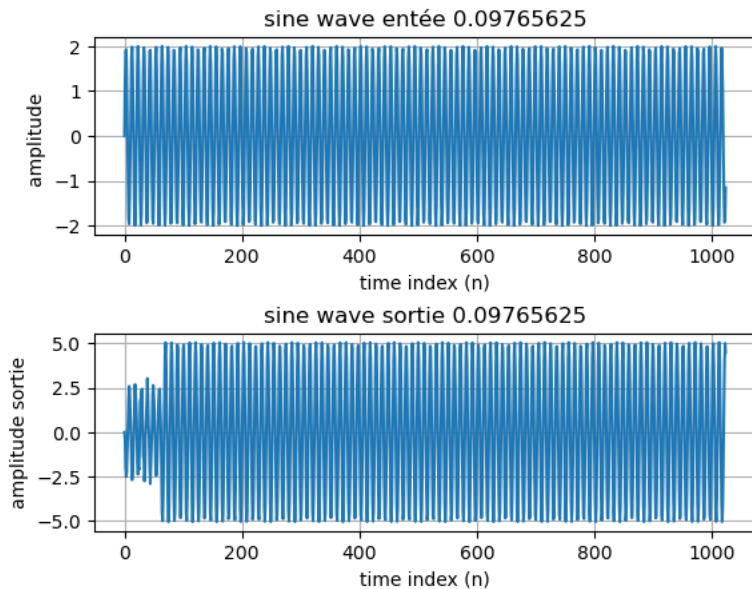
```
#réponse impulsionnelle
M=1024 #number of samples for the simulation
imp=np.zeros(M)
imp[0]=1 #create an impulse signal

plt.figure()
h=sci.lfilter(b,a,imp) #h is the impulse response of this filter
plt.plot(h[:1000]) # here, just the first 100 samples. to have the full
plot plt.plot(h)
plt.xlabel("frequency")
plt.ylabel(" ")
plt.title("impulse response")
plt.grid()
```

▪ réponse à un sinus, essayez plusieurs fréquences. Montrer que vous êtes capable de

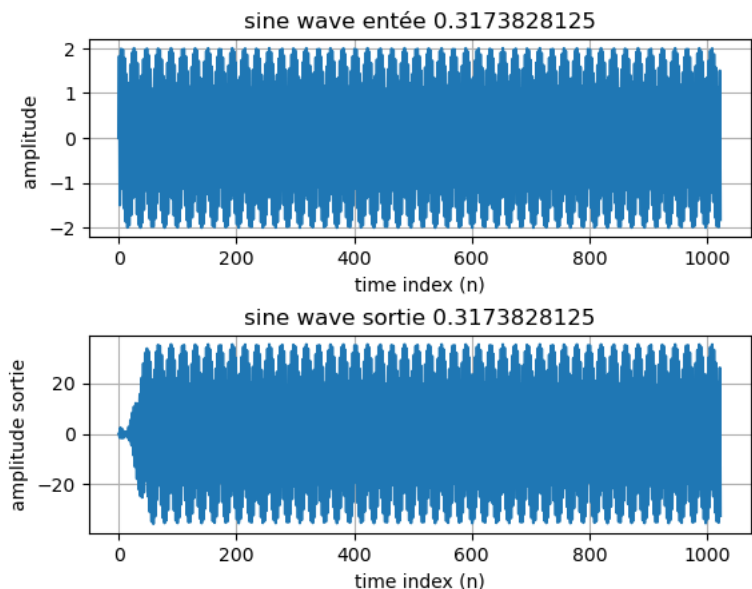
prédire l'amplitude de sortie et vérifier votre résultat en simulation python

max_s = 5.0559



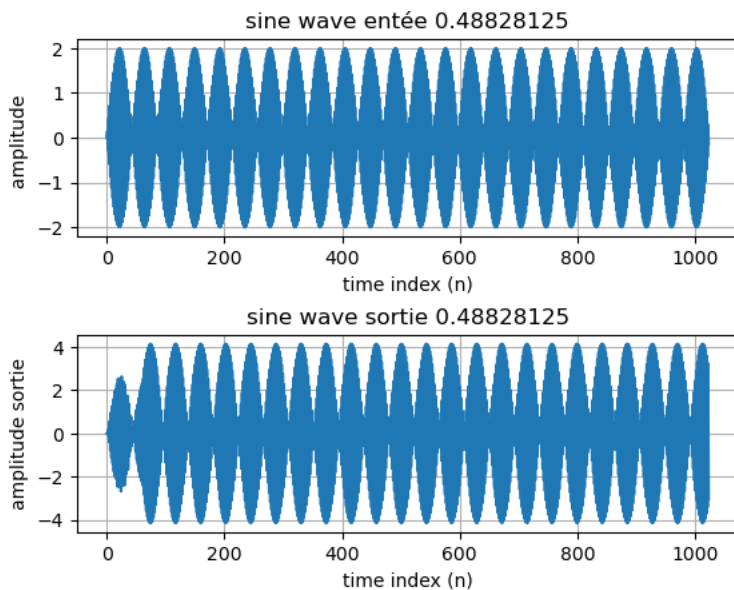
pour une fréquence de **0.0977**, on obtien -theoriquemen une amplitude de sortie de **:5.0563** -expérimentalement une amplitude de sortie de **:5.0559**

max_s = 35.2849



pour une fréquence de **0.3174**, on obtien -theoriquemen une amplitude de sortie de **:35.2851** -expérimentalement une amplitude de sortie de **:35.2849**

max_s = 4.1512



pour une fréquence de **0.4883**, on obtient -théoriquement une amplitude de sortie de **:4.1515** -expérimentalement une amplitude de sortie de **:4.1512**

max_s = 5.0559 max_s = 35.2849 max_s = 4.1512

freq = 0.0977 | Gian = 8.0561 dB | amplitude_filtre = 2.5282 | amplitude_sinus_sortie = 5.0563

freq = 0.3174 | Gian = 24.9312 dB | amplitude_filtre = 17.6426 | amplitude_sinus_sortie = 35.2851

freq = 0.4883 | Gian = 6.3436 dB | amplitude_filtre = 2.0758 | amplitude_sinus_sortie = 4.1515

In []:

```
#réponse à un signal sinusoïdal
fo=[w[200],w[650],w[1000]] #frequency of the sine wave
#n=np.linspace(0,100,M) #time index
n=np.arange(0,M) #time index
print(n)
for i in range(len(fo)):
    x=2*np.sin(2*np.pi*n*fo[i]) #the sine wave
    s=sci.lfilter(b,a,x) #s is the output of the filter
    max_s=np.max(s[64:])
    print(f' max_s = {max_s:.4f} ')
    plt.figure()
    plt.subplot(211)
    plt.plot(n,x)
    plt.xlabel("time index (n)")
    plt.ylabel("amplitude ")
    plt.title(f"sine wave entrée {fo[i]}")
    plt.grid()
```

```

plt.subplot(212)
plt.plot(n,s)
plt.xlabel("time index (n)")
plt.ylabel("amplitude sortie")
plt.title(f"sine wave sortie {fo[i]}")
plt.grid()
#espacer les figures
plt.subplots_adjust(hspace=0.5)
plt.show()

index=200
T=np.abs(H[index])
Gian=20*np.log10(T)
print(f'freq = {w[index]:.4f} | Gian = {Gian:.4f} dB | amplitude_filtre =
{T:.4f} | amplitudde_sinus_sortie = {2*T:.4f}')
```

```

index=650
T=np.abs(H[index])
Gian=20*np.log10(T)
print(f'freq = {w[index]:.4f} | Gian = {Gian:.4f} dB | amplitude_filtre =
{T:.4f} | amplitudde_sinus_sortie = {2*T:.4f}')
```

```

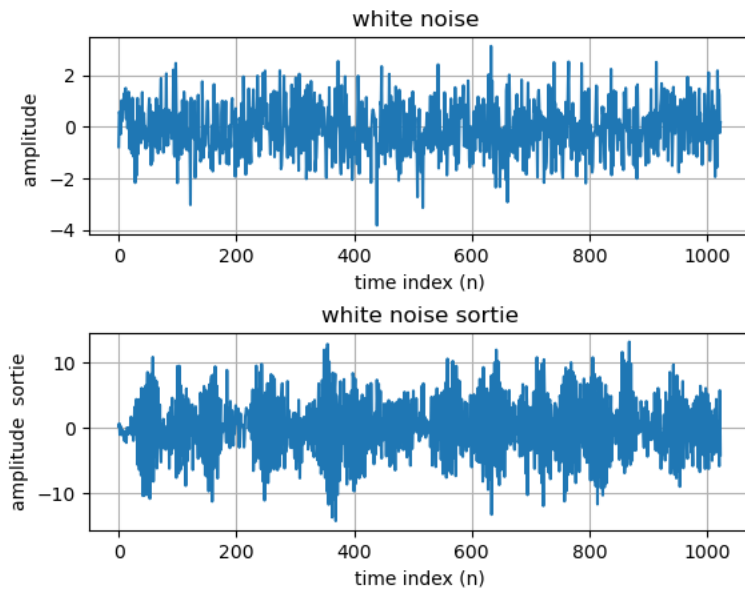
index=1000
T=np.abs(H[index])
Gian=20*np.log10(T)
print(f'freq = {w[index]:.4f} | Gian = {Gian:.4f} dB | amplitude_filtre =
{T:.4f} | amplitudde_sinus_sortie = {2*T:.4f}')
```

```

"""
max_s = 5.0559
max_s = 35.2849
max_s = 4.1512
freq = 0.0977 | Gian = 8.0561 dB | amplitude_filtre = 2.5282 |
amplitudde_sinus_sortie = 5.0563
freq = 0.3174 | Gian = 24.9312 dB | amplitude_filtre = 17.6426 |
amplitudde_sinus_sortie=35.2851
freq = 0.4883 | Gian = 6.3436 dB | amplitude_filtre = 2.0758 |
amplitudde_sinus_sortie = 4.1515
"""
```

▪ Bruit blanc gaussien. Montrez que vous êtes capable de prédire la puissance de sortie

et vérifiez votre résultat en simulation python.



-puissance du bruit en entrée = ***1.0262***

-puissance du bruit en sortie = ***26.5798***

-puissance du bruit en sortie (théorique) = ***23.9998***

In []:

```
#réponse a un bruit blanc gaussien
noise=np.random.randn(M)
y_bruit=sci.lfilter(b,a,noise)

plt.figure()
plt.subplot(211)
plt.plot(noise)
plt.xlabel("time index (n)")
plt.ylabel("amplitude ")
plt.title("white noise ")
plt.grid()

plt.subplot(212)
plt.plot(y_bruit)
plt.xlabel("time index (n)")
plt.ylabel("amplitude sortie")
plt.title("white noise sortie")
plt.grid()
plt.subplots_adjust(hspace=0.5)

#puissance du bruit en sortie
P_bruit=np.mean(noise**2)
print(f'puissance du bruit en entrée = {P_bruit:.4f}')
P_bruit_sortie=np.mean(y_bruit**2)
print(f'puissance du bruit en sortie = {P_bruit_sortie:.4f}')
```

```
# puissance de sortie théorique
P_bruit_theorique=np.sum(b**2)*P_bruit
print(f'puissance de sortie théorique = {P_bruit_theorique:.4f}')
```

codage optimal en virgule fixe pour ces coefficients.

$N_{total} = E + F + 1$

- 1 bit pour le signe (car les coefficient sont signés)
- E bits pour la partie entière
- F bits pour la partie fractionnaire

$coef_{min} = -1.3157415252968885$

$coef_{max} = 0.9031506811907334$

donc

$abs(coef_{min}) = 1.3157415252968885$

$abs(coef_{max}) = 0.9031506811907334$

→ le max est 1.3157415252968885

$\log_2(10) \approx 0.3958761024487547$

$\text{ceil}(0.3958761024487547) \rightarrow 1.0$

integer_bits = 1

This means 1 bit is needed to represent values up to ± 10 .

donc $E = 1.0$

$F = N_{total} - E - 1$

$F = 16 - 1 - 1 = 14$

Fixed-point coefficients: pour arduino

**[-5793 -18755 4594 -1676 1906 -1619 -496 2832 -2327 -1635 5068 -3111 -3597
7837 -3400 -6429 10659 -2805 -9959 13060 -1185 -13749 14549 1394 -17309
14797 4703 -20049 13694 8195 -21557 11360 11360 -21557 8195 13694 -20049**

**4703 14797 -17309 1394 14549 -13749 -1185 13060 -9959 -2805 10659 -6429 -
3400 7837 -3597 -3111 5068 -1635 -2327 2832 -496 -1619 1906 -1676 4594 -
18755 -5793]**

In []:

```
# Determine the range of coefficients
coef_min = np.min(coef)
coef_max = np.max(coef)
print(f'Min coefficient: {coef_min}, Max coefficient: {coef_max}')

# Calculate the number of integer bits required
integer_bits = int(np.ceil(np.log2(max(abs(coef_min), abs(coef_max))))) #
Calculate the number of bits needed to represent the integer part
print(f'Number of integer bits required: {integer_bits}')
# Number of integer bits required: 1
"""
Exemple:
coef_min = -10
coef_max = 8
Then:
abs(coef_min) = 10, abs(coef_max) = 8 → max is 10
log2(10) ≈ 3.32
ceil(3.32) → 4
integer_bits = 4
This means 4 bits are needed to represent values up to ±10."""

# Define the total number of bits for fixed-point representation
total_bits = 16 # Example: using 16-bit fixed-point representation

# Calculate the number of fractional bits
fractional_bits = total_bits - integer_bits - 1 # Subtract 1 for the sign
bit
print(f'Number of fractional bits: {fractional_bits}')

# Calcul du facteur d'échelle
scaling_factor = 2 ** fractional_bits
"""
Exemple : si fractional_bits = 8, alors scaling_factor = 256.
Cela signifie que 0.5 deviendra 128 (0.5 * 256 = 128).
"""
fixed_point_coef = np.round(coef * scaling_factor).astype(int)
print(f'Fixed-point coefficients: pour arduino \n {fixed_point_coef}')
```

Trouver le format de codage optimal des opérands et des opérateurs pour éviter

les dépassements et maximiser la précision

$$y(n) = \sum_{i=0}^{64} b_i * x(n-i)$$

- $b_i : Q(16, 14)$
- $x(n) : Q(16, 6)$
- $\text{mult} = b_i * x(n) : Q(32, 20)$

$$N_{\text{bin}} = \lceil \ln(64) \rceil$$

- $\text{sum} = \sum_{i=0}^{64} \text{mult} : Q(64, 20)$
- $y(n) : Q(16, 6)$

In []:

```
# Étape 1 : Déterminer la plage des coefficients
coef_range = (coef_min, coef_max)
print(f"Plage des coefficients : {coef_range}")

# Étape 2 : Calculer la plage des valeurs intermédiaires (produits)
# Les produits sont calculés entre les coefficients et les valeurs d'entrée
# (par exemple, l'impulsion ou un signal sinusoïdal ou un bruit)
# Supposons que l'entrée est normalisée entre -1 et 1
input_range = (0, 1)
product_min = coef_min * input_range[0]
product_max = coef_max * input_range[1]
product_range = (min(product_min, product_max), max(product_min,
product_max))
print(f"Plage des produits intermédiaires : {product_range}")

# Étape 3 : Calculer la plage des accumulations
# L'accumulation est la somme des produits intermédiaires
# La valeur maximale d'accumulation est obtenue en supposant que tous les
coefficients sont positifs
accumulation_max = np.sum(np.abs(coef))
accumulation_min = -accumulation_max
accumulation_range = (accumulation_min, accumulation_max)
print(f"Plage des accumulations : {accumulation_range}")

# Étape 4 : Déterminer le nombre de bits nécessaires pour éviter les
dépassements
# Le nombre de bits pour la partie entière doit couvrir la plage des
accumulations
integer_bits_accumulation = int(np.ceil(np.log2(max(abs(accumulation_min),
abs(accumulation_max))))))
print(f"Nombre de bits pour la partie entière (accumulation) :
{integer_bits_accumulation}")
```

```

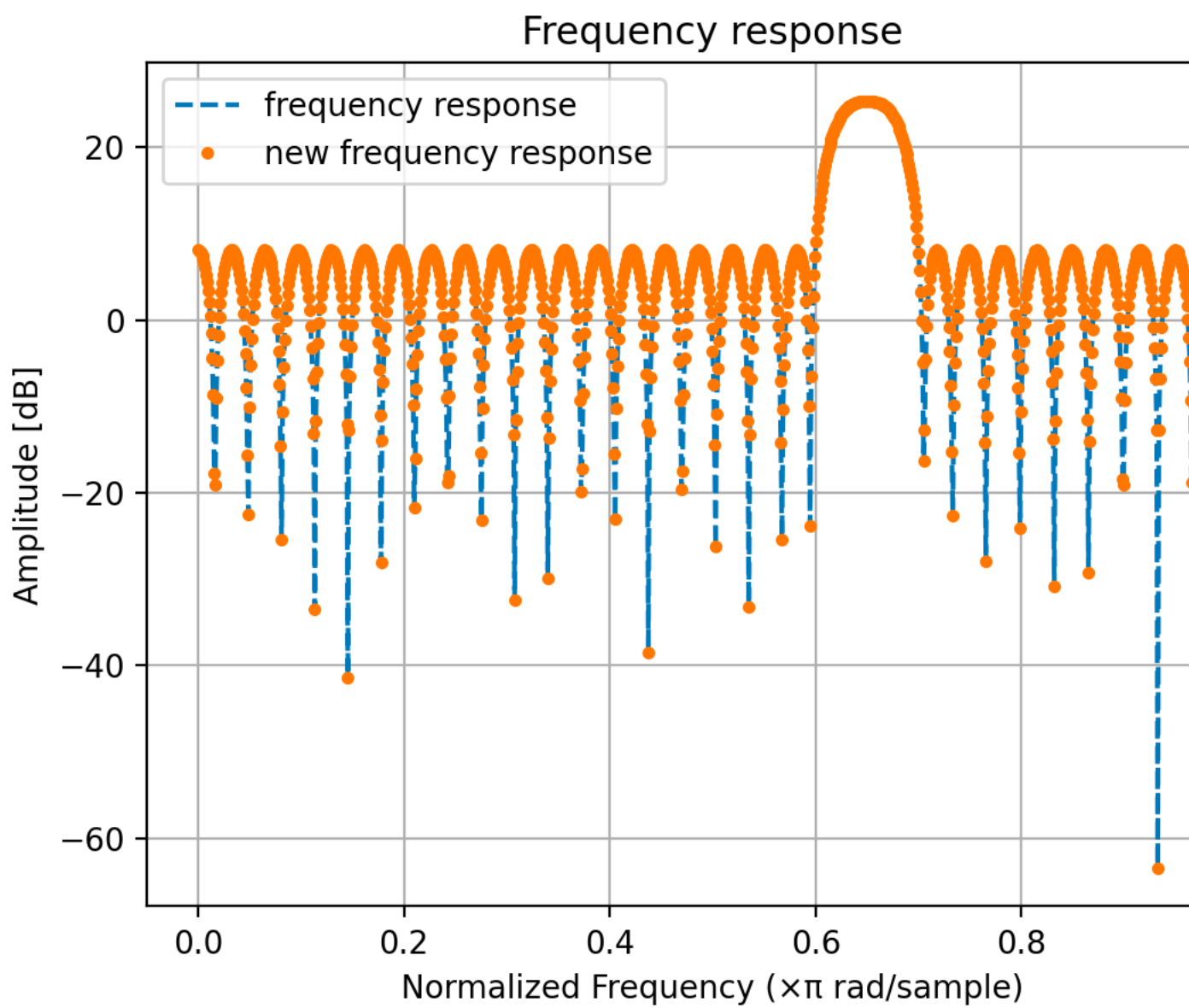
# Étape 5 : Maximiser la précision
# Le nombre total de bits est défini (par exemple, 16 bits)
# Les bits restants après la partie entière sont utilisés pour la partie
fractionnaire
fractional_bits_accumulation = total_bits - integer_bits_accumulation - 1
# -1 pour le bit de signe
print(f"Nombre de bits pour la partie fractionnaire (accumulation) :
{fractional_bits_accumulation}")

# Résumé du format optimal
print(f"Format optimal : {integer_bits_accumulation} bits pour la partie
entière, "
      f"{fractional_bits_accumulation} bits pour la partie fractionnaire.")

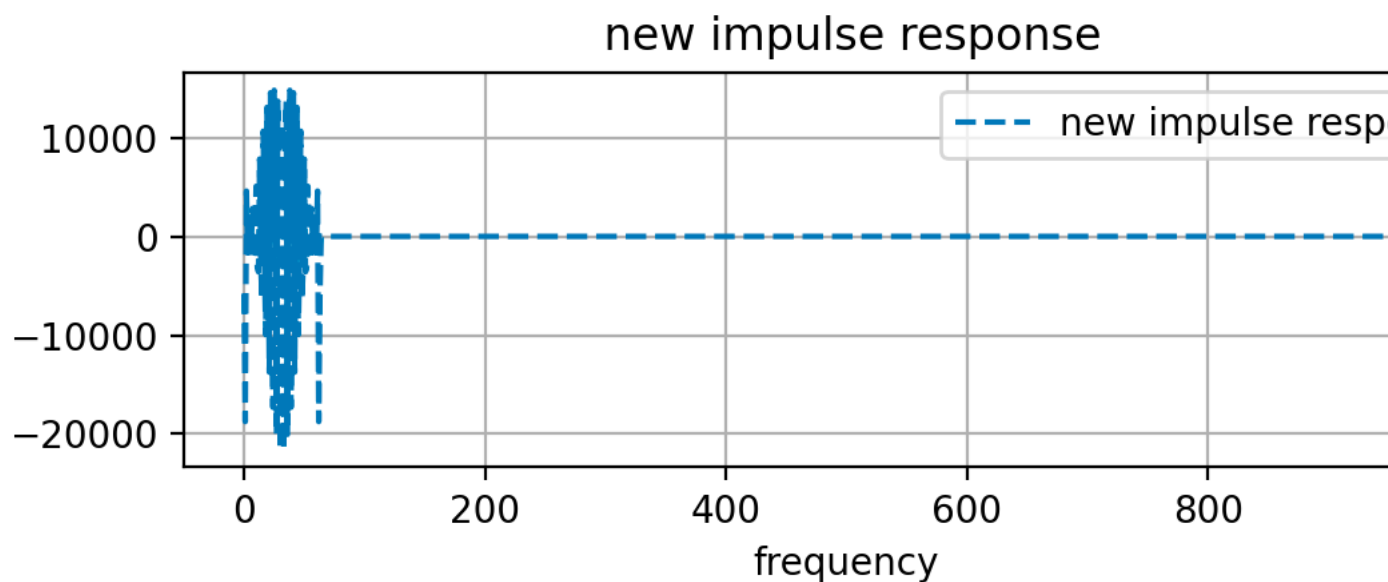
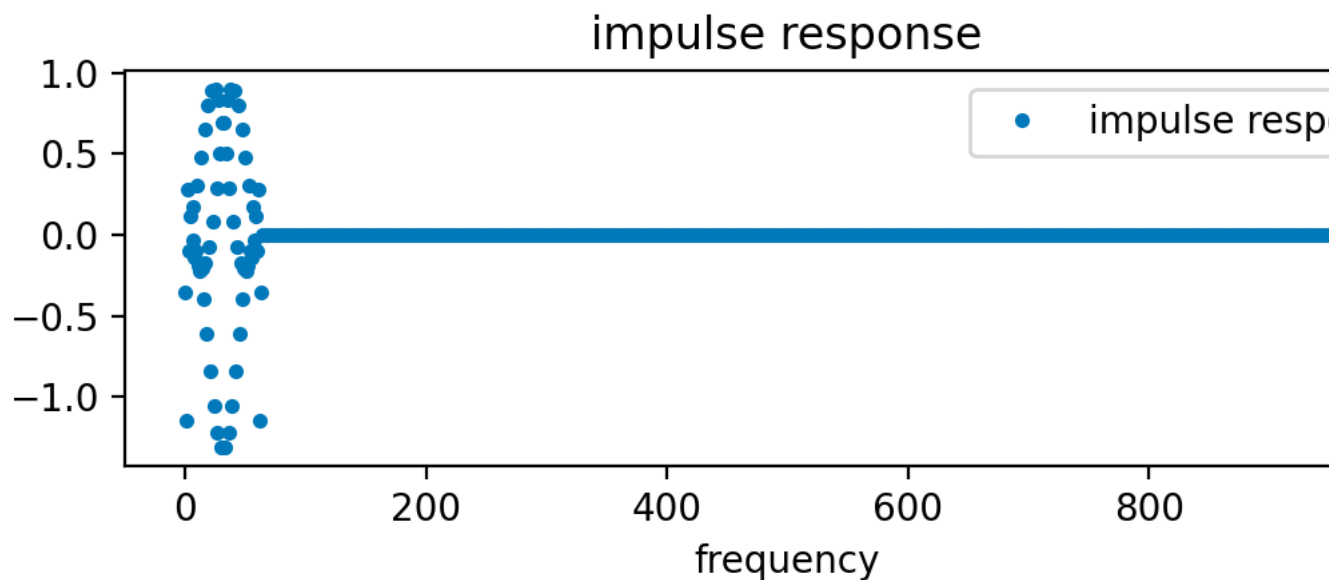
```

Vérification de la réponse impulsionnelle et la
réponse à une fréquence avec les coef implémenté dans arduino.

réponse à une fréquence



réponse impulsionnelle



coef pour arduino

```
b=[ -5793. -18755. 4594. -1676. 1906. -1619. -496. 2832. -2327. -1635. 5068. -3111. -
3597. 7837. -3400. -6429. 10659. -2805. -9959. 13060. -1185. -13749. 14549. 1394. -
17309. 14797. 4703. -20049. 13694. 8195. -21557. 11360. 11360. -21557. 8195. 13694.
-20049. 4703. 14797. -17309. 1394. 14549. -13749. -1185. 13060. -9959. -2805. 10659.
-6429. -3400. 7837. -3597. -3111. 5068. -1635. -2327. 2832. -496. -1619. 1906. -1676.
4594. -18755. -5793.]
```

```
a = 1
```

In []:

```
import numpy as np
import scipy.signal
import matplotlib.pyplot as plt
#%matplotlib qt
```

```

coef=np.loadtxt("coef21.txt", delimiter=',')
b=coef
a=1
#print(b,"a = ",a)
# Point fixed values of the coefficients:
Fc=14
coef = np.round(coef * 2**Fc) / 2**Fc # rounding to 14 bits
b1=coef
a1=1
#print("new  \n",b1,"a = ",a)

# int14_t values of the coefficients:
coef = np.round(coef * 2**Fc) # rounding to 14 bits
b2=coef
a2=1
print("coef pour arduino  \n" ,b2,"a = ",a)

# Plot the frequency response
w, h = scipy.signal.freqz(b, a, worN=1024)
w1, h1 = scipy.signal.freqz(b1, a, worN=1024)

plt.figure()
plt.plot(w/np.pi, 20 * np.log10(abs(h)), '--', label="frequency response") #
Magnitude in dB
plt.plot(w1/np.pi, 20 * np.log10(abs(h1)), '.', label="new frequency
response") # Magnitude in dB
plt.title('Frequency response')
plt.xlabel('Normalized Frequency ( $\pi$  rad/sample)')
plt.ylabel('Amplitude [dB]')
plt.grid()
plt.legend()
plt.show()

#réponse impulsionnelle
M=1024 #number of samples for the simulation
imp=np.zeros(M)
imp[0]=1 #create an impulse signal
h=sci.lfilter(b,a,imp)
h2=sci.lfilter(b2,a,imp) #h is the impulse response of this filter

plt.figure()
plt.subplot(211)
plt.plot(h[:1000], '.', label="impulse response") # here, just the first 100
samples. to have the full plot plt.plot(h)
plt.title("impulse response")
plt.xlabel("frequency")

```

```
plt.legend()
plt.subplot(212)
plt.plot(h2[:1000], '--', label="new impulse response")
plt.title(" new impulse response")
plt.xlabel("frequency")
plt.ylabel(" ")
plt.grid()
plt.legend()
plt.subplots_adjust(hspace=0.5)
plt.show()
```

Envoi d'une suite d'échantillons sur la liaison série de l'Arduino,
traitement l'échantillon dans l'arduino puis renvoi de la valeur traitée
par liaison série au programme python.

In []:

```
# === CONFIGURATION ===

t=np.arange(M)
PORT = '/dev/cu.usbmodem143301'
BAUDRATE = 115200
#N_SAMPLES = M

# === INITIALISATION SERIAL ===
ser = serial.Serial(PORT, BAUDRATE, timeout=1)
time.sleep(2) # laisser le temps à l'Arduino de démarrer

# create the impulse signal
imp=np.zeros(M)
imp[0]=1

# === GÉNÉRER SIGNAL ===
#t = np.arange(N_SAMPLES)
freq = 0.4
amplitude = 3
#signal=(imp).astype(np.int16)
#signal
signal = (amplitude * np.cos(2 * np.pi * freq * t)).astype(np.int16)
#noise
#bruit = np.random.normal(0, 10, N_SAMPLES).astype(np.int16)
#signal = signal + bruit
#signal = signal.astype(np.int16)
# === ENVOI & RÉCEPTION ===
output = []
for x in signal:
    # envoyer en little endian
```

```

ser.write(struct.pack('<h', x))

# lire réponse filtrée
response = ser.read(2)
if len(response) == 2:
    yn = struct.unpack('<h', response)[0]
    output.append(yn)
else:
    output.append(0) # ou None si besoin
"""
#frequency response of the filter
HH=np.fft.fft(output)
f,HH=sci.freqz(b1, a1, worN=M, fs=1)"""

# === AFFICHAGE ===
plt.figure()
plt.title("Filtrage FIR temps réel via Arduino")
plt.subplot(211)
plt.plot(signal, label='Entrée (x[n])')
plt.xlabel("temps")
plt.ylabel("x[n]")
plt.legend()
plt.grid()
plt.subplots_adjust(hspace=0.5)

plt.subplot(212)
plt.plot(output, label='Sortie filtrée (y[n])')
plt.xlabel("temps")
plt.ylabel("y[n]")
plt.legend()
plt.grid()
plt.subplots_adjust(hspace=0.5)

"""
plt.subplot(3,1,3)
plt.plot(f, 10*np.log10(np.abs(HH)**2), label="frequency response")
plt.xlabel("normalized frequency")
plt.ylabel("psd [dB]")
plt.legend()
plt.grid()"""

# === FIN ===
ser.close()

```

implémentation du filtre en arduino avec un signal d'entrée généré par un GBF (enplitude 5volts avec une fréquence de 100 Hz)

->En vert c'ets l'entrée : GBF 5V.

->En rose c'est la sortie : après passage dans le filtre

